

Kreacijski paterni u StudyBuddy sistemu

1. Uvod

StudyBuddy sistem omogućava korisnicima kreiranje sesija učenja, prijavljivanje na sesije, evidentiranje prisustva, pregled obavještenja i upravljanje osnovnim podacima sistema. U prethodnim projektnim zadacima kreiran je osnovni klasni dijagram koji prikazuje glavne klase sistema, kao što su **Korisnik**, **SesijaUcenja**, **PrijavaNaSesiju**, **Prisustvo**, **Predmet**, **Lokacija**, **Uloga** i **Obavjestenje**.

Kako sistem postaje složeniji, javlja se potreba za boljom organizacijom procesa kreiranja objekata. Kreacijski paterni omogućavaju da se objekti ne kreiraju direktno na više različitih mjesta u sistemu, nego kroz posebne klase koje preuzimaju odgovornost za njihovo pravilno kreiranje.

U StudyBuddy sistemu odabrana su dva kreacijska paterna: **Factory Method** i **Builder**.

2. Potreba za dodavanjem kreacijskih paterna

U postojećem sistemu određeni objekti imaju složen proces kreiranja. Na primjer, klasa **Obavjestenje** koristi se u više različitih situacija: prilikom prijave korisnika na sesiju, otkazivanja sesije, promjene termina ili slanja podsjetnika. Iako se u svim slučajevima radi o obavještenju, naslov, sadržaj i tip obavještenja nisu isti.

Ako bi se obavještenja kreirala direktno u više različitih dijelova sistema, kod bi se ponavljao i bilo bi teže dodati nove vrste obavještenja. Zbog toga se uvodi **Factory Method** patern, koji centralizuje kreiranje obavještenja.

Druga situacija odnosi se na kreiranje objekta klase **SesijaUcenja**. Ova klasa ima više atributa, kao što su naziv, opis, datum i vrijeme, trajanje, lokacija, predmet, kreator i maksimalan broj učesnika. Kreiranje takvog objekta direktno kroz konstruktor ili ručnim postavljanjem atributa može postati nepregledno.

Zbog toga se uvodi **Builder** patern, koji omogućava postepeno i kontrolisano kreiranje sesije učenja.

3. Moguća primjena kreacijskih paterna u StudyBuddy sistemu

Kreacijski paterni omogućavaju fleksibilnije, preglednije i sigurnije kreiranje objekata. Njihova primjena u StudyBuddy sistemu može pomoći u smanjenju ponavljanja koda, lakšem dodavanju novih funkcionalnosti i boljoj organizaciji složenih procesa kreiranja objekata.

Singleton patern

Singleton patern se koristi kada sistem treba imati samo jednu instancu određene klase. U StudyBuddy sistemu ovaj patern bi se mogao koristiti za klase koje predstavljaju zajedničke servise, na primjer servis za konfiguraciju aplikacije ili servis za slanje sistemskih obavještenja.

Međutim, u ASP.NET MVC aplikaciji mnogi servisi se već registruju i upravljaju kroz dependency injection mehanizam, pa eksplicitno dodavanje Singleton paterna u klasni dijagram nije najprirodniji izbor za trenutni sistem.

Factory Method patern

Factory Method patern je prikladan kada sistem treba kreirati objekte iste osnovne klase, ali u različitim varijantama. U StudyBuddy sistemu ovaj patern se može primijeniti na kreiranje obavještenja.

Sistem koristi obavještenja u više različitih situacija, kao što su prijava na sesiju, promjena termina, podsjetnik i otkazivanje. Umjesto da se svako obavještenje ručno kreira u različitim dijelovima sistema, uvodi se klasa ObavjestenjeFactory, koja kreira objekte koji implementiraju interfejs IObavjestenje. U zavisnosti od tipa obavještenja kreira se jedna od konkretnih klasa:

- PrijavaObavjestenje
- OtkazivanjeObavjestenje
- PodsjetnikObavjestenje
- PromjenaTerminaObavjestenje

Ovaj patern je posebno koristan jer omogućava jednostavno dodavanje novih tipova obavještenja bez velikih izmjena u postojećim klasama.

Abstract Factory patern

Abstract Factory patern bi se mogao koristiti kada bi sistem imao više povezanih grupa objekata koje se kreiraju zajedno. Na primjer, ako bi StudyBuddy u budućnosti podržavao različite module za online i fizičke sesije, mogla bi postojati posebna fabrika za kreiranje objekata vezanih za online sesije i posebna fabrika za fizičke sesije.

Ipak, za trenutnu verziju sistema ovaj patern nije izabran jer bi nepotrebno zakomplikovao model.

Builder patern

Builder patern se koristi kada je potrebno kreirati složen objekat koji ima više atributa. U StudyBuddy sistemu ovaj patern se može primijeniti na klasu **SesijaUcenja**.

Sesija učenja sadrži više povezanih podataka: naziv, opis, datum i vrijeme, trajanje, predmet, lokaciju, kreatora i maksimalan broj učesnika. Builder omogućava da se ovi podaci postavljaju postepeno, kroz jasne metode, umjesto da se svi podaci šalju kroz jedan veliki konstruktor. Zbog toga je Builder prirodan izbor za kreiranje objekata klase **SesijaUcenja**.

Prototype patern

Prototype patern se koristi kada se novi objekti kreiraju kopiranjem postojećih objekata. U StudyBuddy sistemu ovaj patern bi se mogao koristiti ako bi korisnici često kreirali slične sesije učenja, na primjer sedmične sesije za isti predmet. Međutim, pošto trenutni sistem ne zahtijeva kopiranje postojećih sesija, Prototype nije izabran za finalni dijagram.

4. Odabrani kreacijski paterni

Factory Method patern

Factory Method patern se koristi kada je potrebno centralizovati proces kreiranja objekata, posebno kada postoje različite varijante istog tipa objekta.

U StudyBuddy sistemu ovaj patern je primijenjen kroz klasu **ObavjestenjeFactory**. Ova klasa omogućava kreiranje obavještenja na osnovu tipa obavještenja, korisnika i sesije učenja.

Klasa **ObavjestenjeFactory** koristi sljedeće klase:

- **IObavjestenje**
- **PrijavaObavjestenje**
- **OtkazivanjeObavjestenje**
- **PodsjetnikObavjestenje**
- **PromjenaTerminaObavjestenje**
- **Korisnik**
- **SesijaUcenja**
- **TipObavjestenja**

U dijagramu je u klasi **ObavjestenjeFactory** definisana metoda:

+ kreirajObavjestenje(tip : TipObavjestenja, korisnik : Korisnik, sesija : SesijaUcenja) : IObvjestenje

Ova metoda na osnovu proslijeđenog tipa kreira odgovarajući objekat koji implementira interfejs IObvjestenje. Na primjer, ako je tip obavještenja Prijava, kreira se objekat klase PrijavaObavjestenje. Ako je tip Otkazivanje, kreira se objekat klase OtkazivanjeObavjestenje. Ako je tip Podsjetnik, kreira se objekat klase PodsjetnikObavjestenje. Ako je tip PromjenaTermina, kreira se objekat klase PromjenaTerminaObavjestenje.

Prednost ovog paterna je u tome što se kreiranje obavještenja odvaja od ostatka sistema. Klase poput **SesijaFacade** ne moraju znati detalje o tome kako se formira naslov i sadržaj obavještenja, nego samo koriste fabriku za kreiranje odgovarajućeg objekta.

Builder patern

Builder patern se koristi kada objekat ima više atributa i kada njegovo kreiranje treba biti pregledno i kontrolisano.

U StudyBuddy sistemu ovaj patern je primijenjen kroz klasu **SesijaUcenjaBuilder**. Ova klasa omogućava postepeno kreiranje objekta klase SesijaUcenja.

Klasa **SesijaUcenjaBuilder** koristi sljedeće klase:

- SesijaUcenja
- Korisnik
- Predmet
- Lokacija

U dijagramu su u klasi **SesijaUcenjaBuilder** definisane metode:

+ postaviNaziv(naziv : string) : SesijaUcenjaBuilder
+ postaviOpis(opis : string) : SesijaUcenjaBuilder
+ postaviDatumVrijeme(datumVrijeme : DateTime) : SesijaUcenjaBuilder
+ postaviTrajanje(trajanje : int) : SesijaUcenjaBuilder
+ postaviPredmet(predmet : Predmet) : SesijaUcenjaBuilder
+ postaviLokaciju(lokacija : Lokacija) : SesijaUcenjaBuilder
+ postaviKreatora(korisnik : Korisnik) : SesijaUcenjaBuilder
+ postaviMaksimalanBrojUcesnika(broj : int) : SesijaUcenjaBuilder
+ kreiraj() : SesijaUcenja

Svaka metoda vraća objekat tipa **SesijaUcenjaBuilder**, što omogućava lančano pozivanje metoda. Na kraju se poziva metoda `kreiraj()`, koja vraća potpuno formiran objekat klase **SesijaUcenja**.

Prednost ovog paterna je u tome što se kreiranje sesije čini preglednijim i lakšim za održavanje. Ako se u budućnosti doda novi atribut u klasu **SesijaUcenja**, dovoljno je dopuniti builder, bez velikih izmjena u ostatku sistema.

5. Veza između odabranih paterna

U kreiranom dijagramu oba paterna su povezana sa postojećom logikom sistema. Klasa **SesijaFacade**, koja je ranije uvedena kao strukturalni patern, može koristiti oba kreacijska paterna.

SesijaFacade koristi **SesijaUcenjaBuilder** prilikom kreiranja nove sesije učenja, jer builder omogućava postepeno postavljanje svih potrebnih podataka sesije.

Također, **SesijaFacade** koristi **ObavjestenjeFactory** kada je potrebno kreirati obavještenje nakon kreiranja sesije, prijave korisnika na sesiju ili otkazivanja sesije.

Na ovaj način se postiže bolja organizacija sistema:

- Builder pojednostavljuje kreiranje objekta **SesijaUcenja**.
- Factory Method centralizuje kreiranje objekata klase **Obavjestenje**.
- **SesijaFacade** ne mora direktno kreirati složene objekte, nego koristi posebne klase za njihovo kreiranje.
- Sistem postaje fleksibilniji i lakši za proširenje.

6. Zaključak

Dodavanjem Factory Method i Builder paterna **StudyBuddy** sistem postaje pregledniji, fleksibilniji i lakši za održavanje. Factory Method patern omogućava centralizovano kreiranje različitih tipova obavještenja, dok Builder patern omogućava postepeno i kontrolisano kreiranje složenih objekata klase **SesijaUcenja**.

Ovi paterni ne mijenjaju osnovni model sistema, nego ga nadograđuju dodatnim klasama koje poboljšavaju proces kreiranja objekata. Zbog toga su Factory Method i Builder prikladni kreacijski paterni za **StudyBuddy** sistem.

Dokument za strukturne paterne je imao isti pristup: prvo je objašnjena potreba za paternima, zatim moguća primjena više paterna, pa tek onda odabrani paterni i njihova veza sa sistemom

